VULNERABILITY ASSESSMENT & PENETRATION TESTING REPORT

Sustainability Compliance Monitor

Topic	Final Demo
Name	Shrusti Kolalagi
USN	1BY22EC100
College	BMS Institute of Technology & Management
Course	Cybersecurity & Ethical Hacking
Date	

1. Executive Summary

A comprehensive Vulnerability Assessment and Penetration Testing (VAPT) engagement was conducted against an AI-powered Compliance Platform deployed locally on 127.0.0.1. The assessment was performed between May 1–5, 2026 and covered four phases: Network Reconnaissance (Nmap), Vulnerability Scanning (Nessus Essentials), Web Application Testing (Burp Suite & OWASP ZAP), and Database Injection Testing (SQLMap).

The assessment identified a total of 12 distinct findings spanning Critical, High, Medium, and Low severity levels. The most pressing concerns involve an unsupported Python runtime, an unpatched Brotli DoS library, missing security headers, server version disclosure, and broken authentication on API endpoints.

Overall Risk Rating

Severity	Count	CVSS Range	Priority
Critical	1	9.0 – 10.0	P0 – Immediate
High	1	7.0 – 8.9	P1 – Urgent
Medium	5	4.0 – 6.9	P2 – Important
Low	5	0.1 – 3.9	P3 – Standard
Total	12	—	—

Key Recommendations

- Immediate (0–24 hrs): Upgrade Python from 3.9.25 to a supported version (3.12+)
- Short-term (1–7 days): Upgrade Brotli library, patch nginx and Log4j, obtain valid SSL certificate
- Medium-term (1–4 weeks): Implement CSP headers, X-Content-Type-Options, fix broken JWT authentication on all API endpoints
- Long-term (1–3 months): Establish a secure SDLC with automated SAST/DAST in CI/CD pipeline

2. Scope & Methodology

2.1 Scope

Component	Details
Target IP	127.0.0.1 (localhost)
Ports In-Scope	3000, 5000, 5432, 8080
Applications	Frontend (nginx), AI Service (Werkzeug/Python), Backend API (Tomcat), PostgreSQL DB
Assessment Type	VAPT
Standards	OWASP Top 10, OWASP Testing Guide, NIST SP 800-115

2.2 Out of Scope

- Denial of Service (DoS) attacks
- Social engineering attacks
- Physical security testing
- Production data manipulation

2.3 Testing Phases & Tools

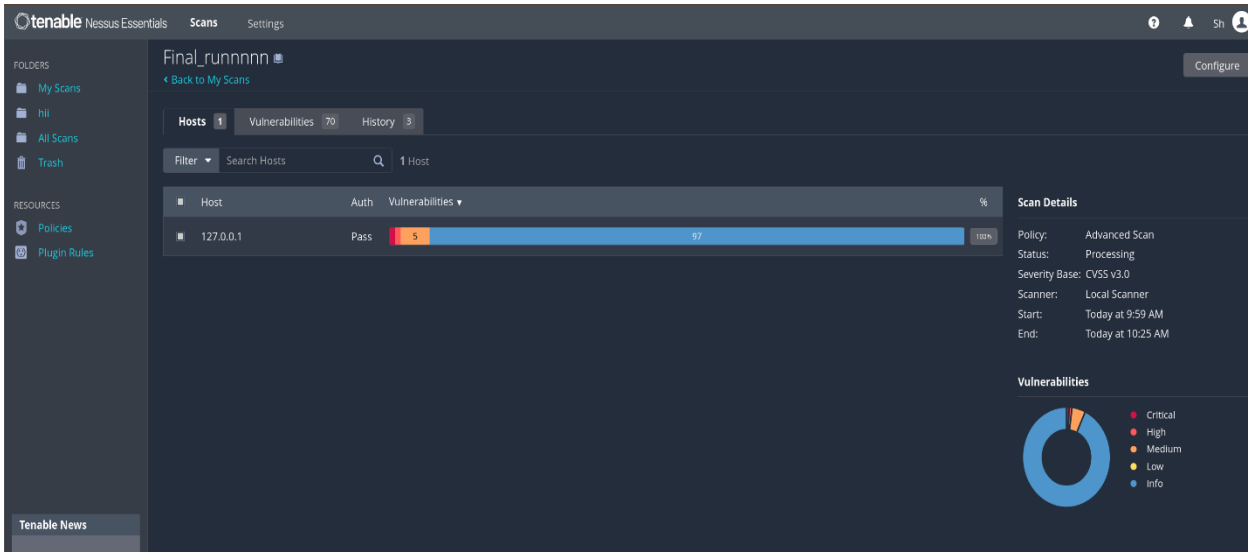
Phase	Tool	Activity
Phase 1	Nmap v7.95	Network reconnaissance, port & service enumeration
Phase 2	Nessus Essentials	Automated vulnerability scanning (CVSS v3.0)
Phase 3A	Burp Suite CE v2025.10.6	Manual web app & API security testing
Phase 3B	OWASP ZAP v2.17.0	Passive scanning & header analysis
Phase 4	SQLMap v1.9.11	SQL injection testing on API endpoints

PHASE 1 : NMAP

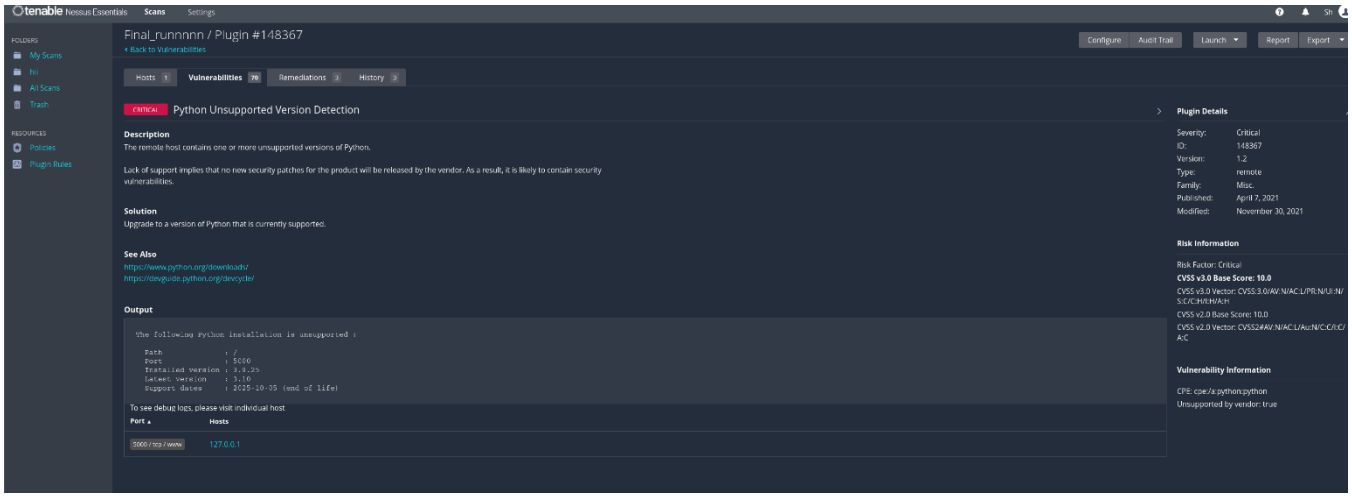
```
┌──(kali㉿kali)-[~]
└─$ nmap -sV 127.0.0.1 -oN ~/Desktop/Phase1_nmap_fullapp.txt
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-04 08:43 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000015s latency).
Not shown: 996 closed tcp ports (reset)
PORT     STATE SERVICE    VERSION
3000/tcp open  http       nginx 1.29.8
5000/tcp open  http       Werkzeug httpd 3.1.8 (Python 3.9.25)
5432/tcp open  postgresql PostgreSQL DB 9.6.0 or later
8080/tcp open  http       Apache Tomcat (language: en)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.09 seconds
```

An Nmap service version scan (nmap -sV) was run against 127.0.0.1. The scan discovered 4 open ports: port 3000 running nginx 1.29.8 (frontend), port 5000 running Werkzeug/Python 3.9.25 (AI service), port 5432 running PostgreSQL 9.6+ (database), and port 8080 running Apache Tomcat (backend API). This confirms the full application stack is exposed on localhost and provides the attacker with precise version information to identify known CVEs.

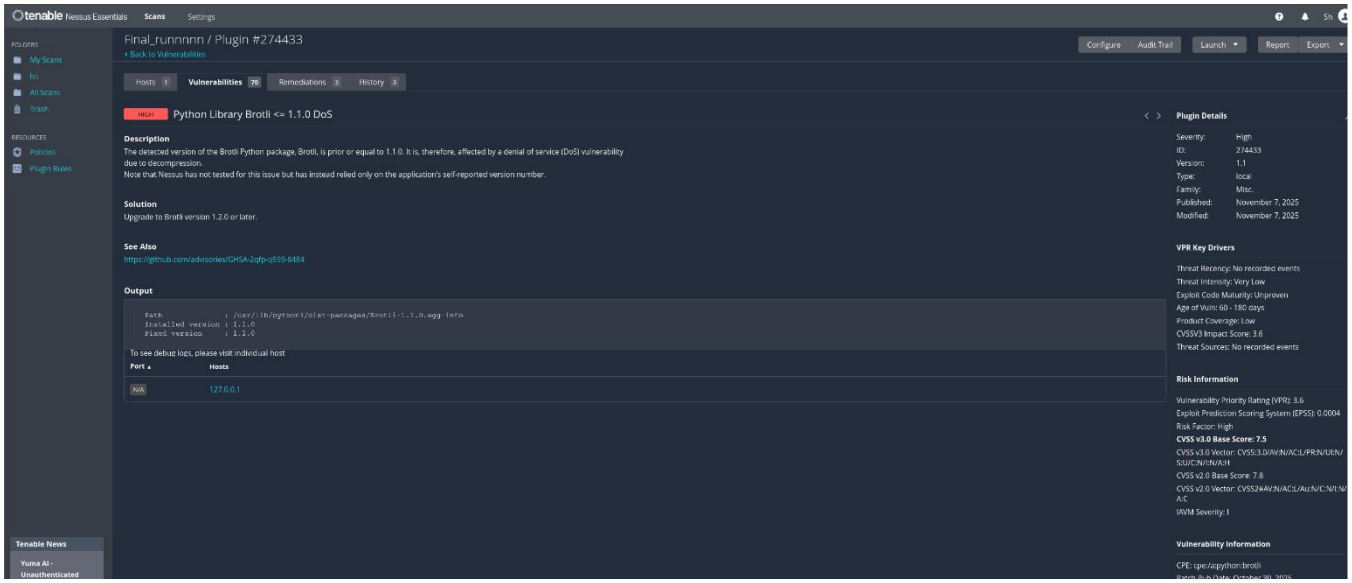
PHASE 2 : NESSUS



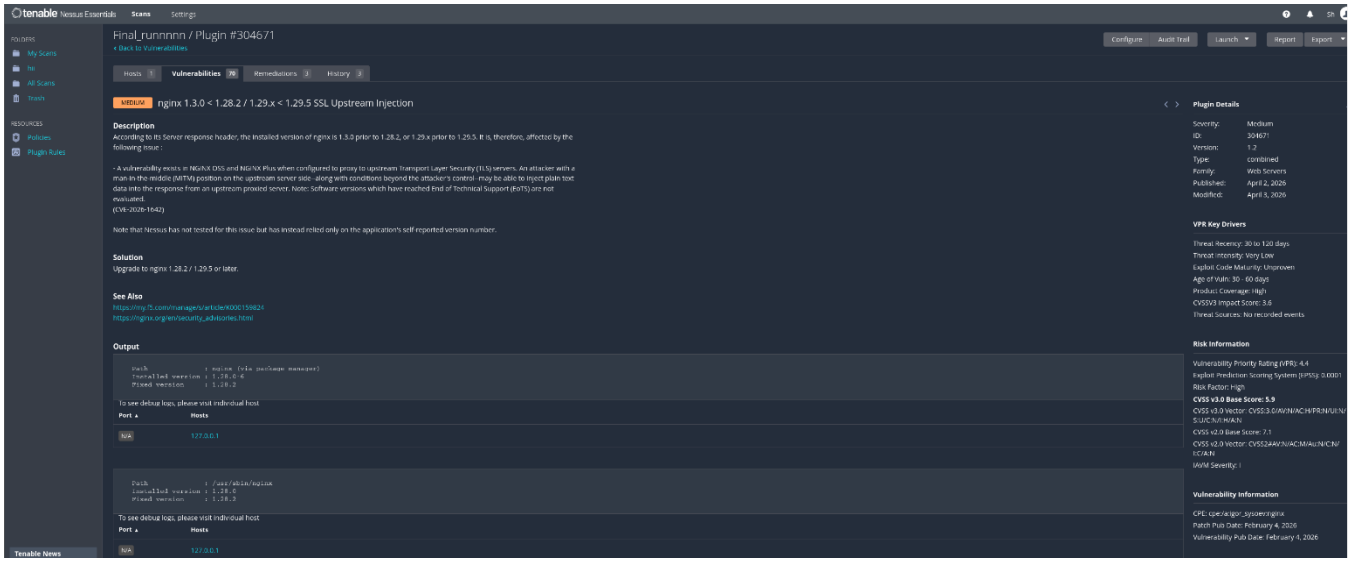
This is the Nessus Essentials scan dashboard for the host 127.0.0.1. The scan ran under the "Advanced Scan" policy using CVSS v3.0 scoring and completed between 9:59 AM and 10:25 AM. It identified a total of 70 vulnerabilities across the host, with the donut chart showing a breakdown including Critical and Medium severity issues. The 5 colored segments (red/orange) indicate high-priority findings that require immediate attention.



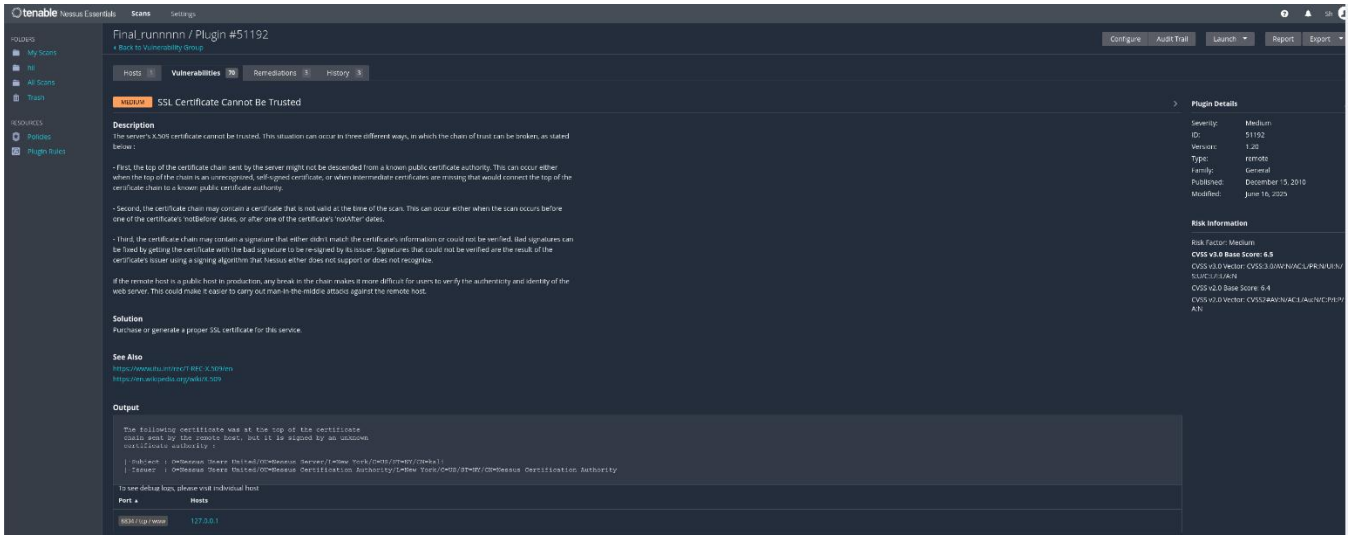
Nessus Plugin #148367 flagged a Critical severity finding — Python Unsupported Version Detection. The installed Python version is 3.9.25 running on port 5000, which reached End of Life on October 5, 2025. Since no further security patches will be released for this version, any newly discovered Python vulnerability will remain permanently unpatched, resulting in a maximum CVSS v3.0 score of 10.0.



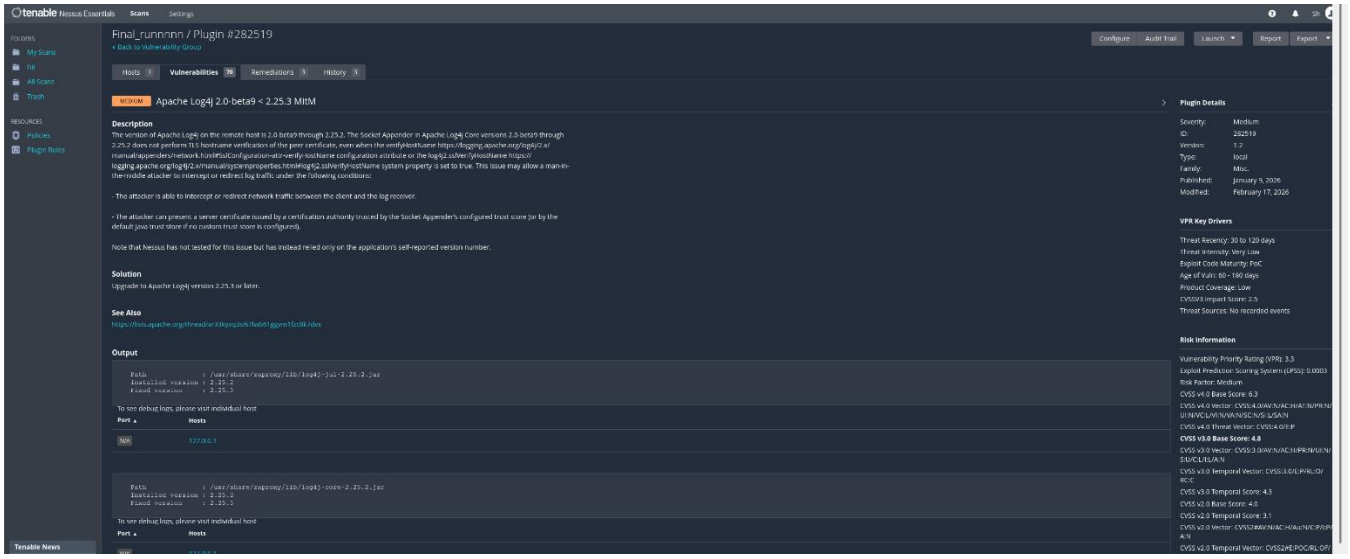
Nessus Plugin #274433 identified a High severity vulnerability in the Brotli Python compression library. The installed version 1.1.0 (found at /usr/lib/python3/dist-packages/) is affected by a denial-of-service vulnerability via decompression (GHSA-2qfp-q593-8484). The fixed version is 1.2.0 and the CVSS v3.0 score is 7.5, meaning an unauthenticated remote attacker can crash the service by sending specially crafted compressed data.



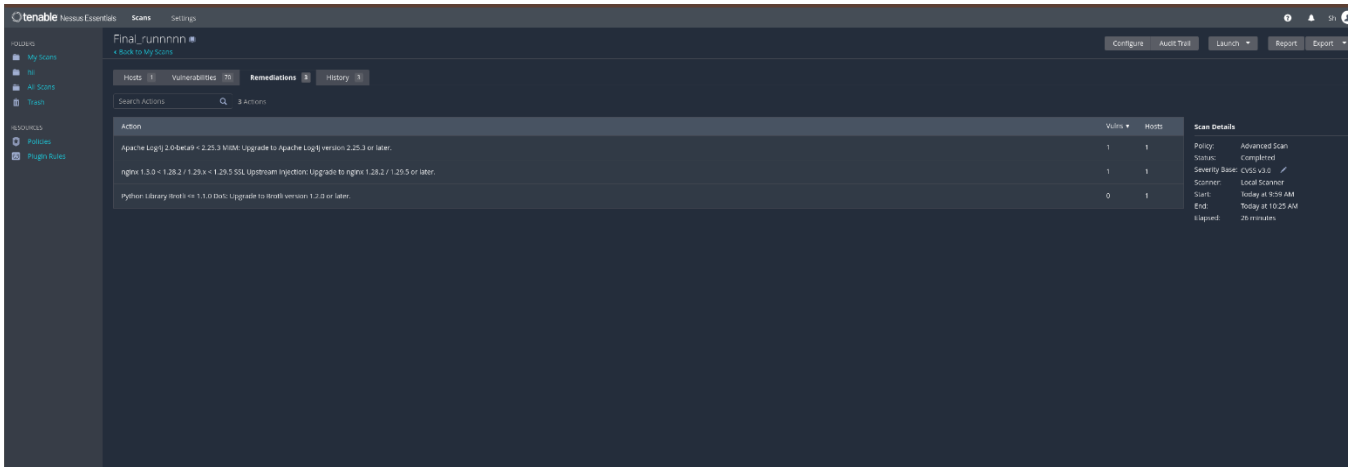
Nessus Plugin #304671 detected a Medium severity vulnerability in the nginx installation. Two instances of nginx version 1.28.0 were found — one installed via package manager and one at /usr/sbin/nginx — both below the patched version 1.28.2. The vulnerability (CVE-2026-1642) allows a man-in-the-middle attacker positioned between nginx and an upstream TLS server to inject plain text data into proxied responses, with a CVSS v3.0 score of 5.9.



Nessus Plugin #51192 flagged a Medium severity finding — SSL Certificate Cannot Be Trusted — on port 8834. The certificate presented is self-signed and issued by "Nessus Certification Authority" rather than a publicly trusted CA. This means clients cannot verify the server's identity, making the connection vulnerable to man-in-the-middle attacks. The CVSS v3.0 score is 6.5 and the fix is to replace the self-signed certificate with one from a trusted CA.

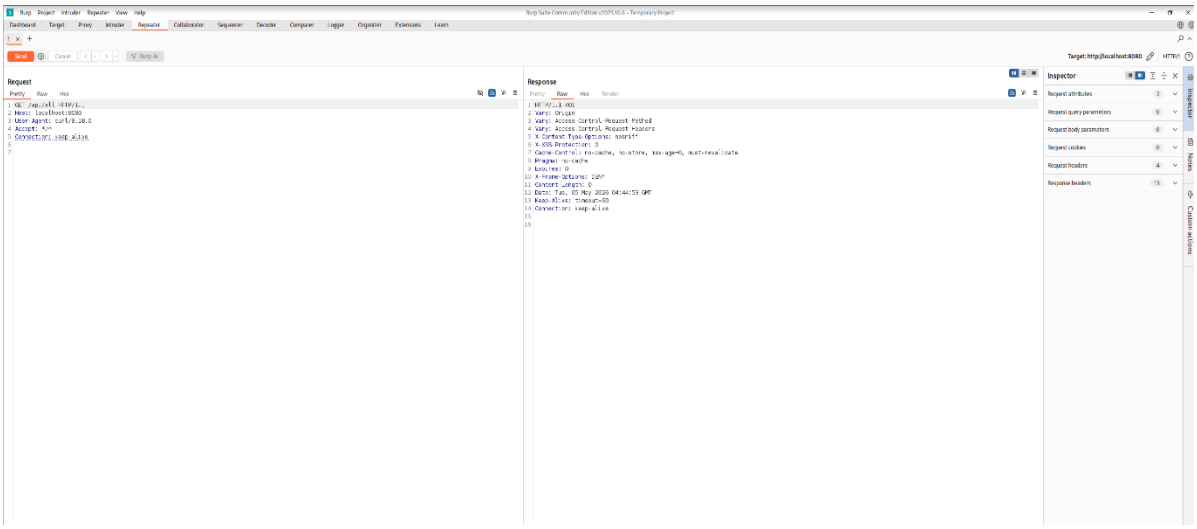


Nessus Plugin #282519 identified a Medium severity vulnerability in Apache Log4j. Two instances of log4j version 2.25.2 were found at /usr/share/zaproxy/lib/ (bundled with the ZAP tool). Versions 2.0-beta9 through 2.25.2 fail to perform TLS hostname verification in the Socket Appender, allowing a man-in-the-middle attacker to intercept log traffic. The CVSS v3.0 score is 4.8 and the fix is to upgrade to Log4j 2.25.3 or later.

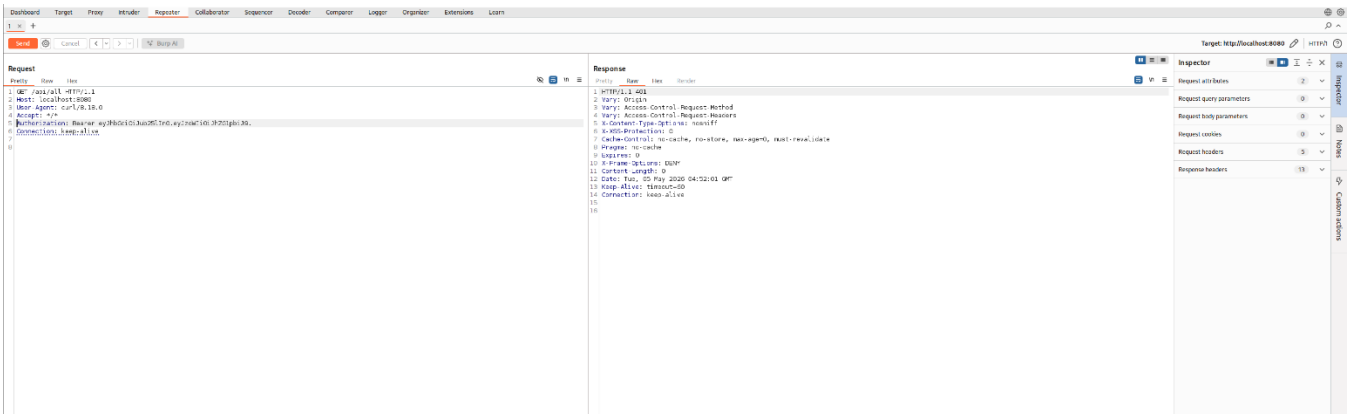


This is the Nessus Remediations tab summarizing the 3 actionable remediation steps identified during the scan. The three actions are: upgrade Apache Log4j to 2.25.3+, upgrade nginx to 1.28.2/1.29.5+, and upgrade Brotli to 1.2.0+. The scan completed in 26 minutes with status "Completed", confirming the scan ran fully and all findings were captured.

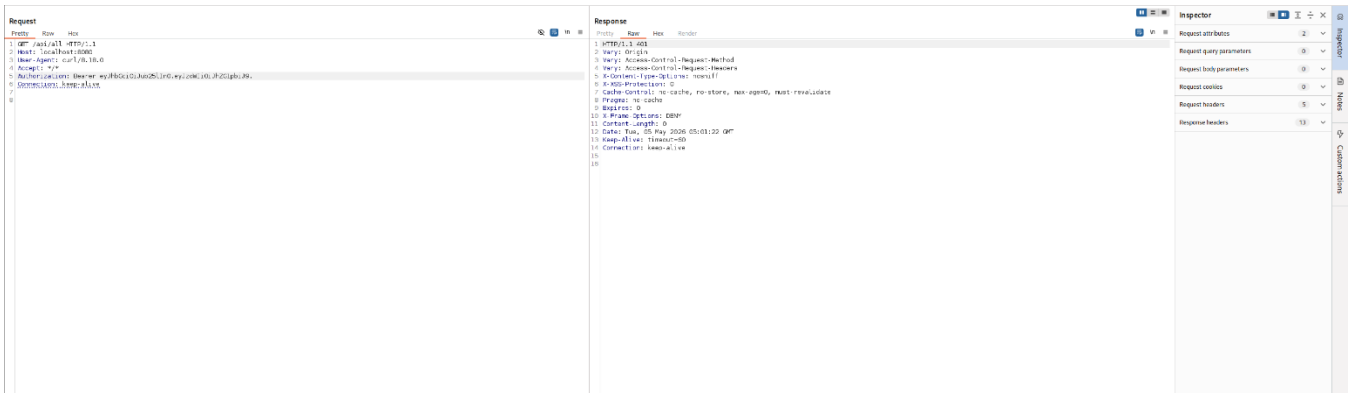
PHASE 3 : BURPSUITE



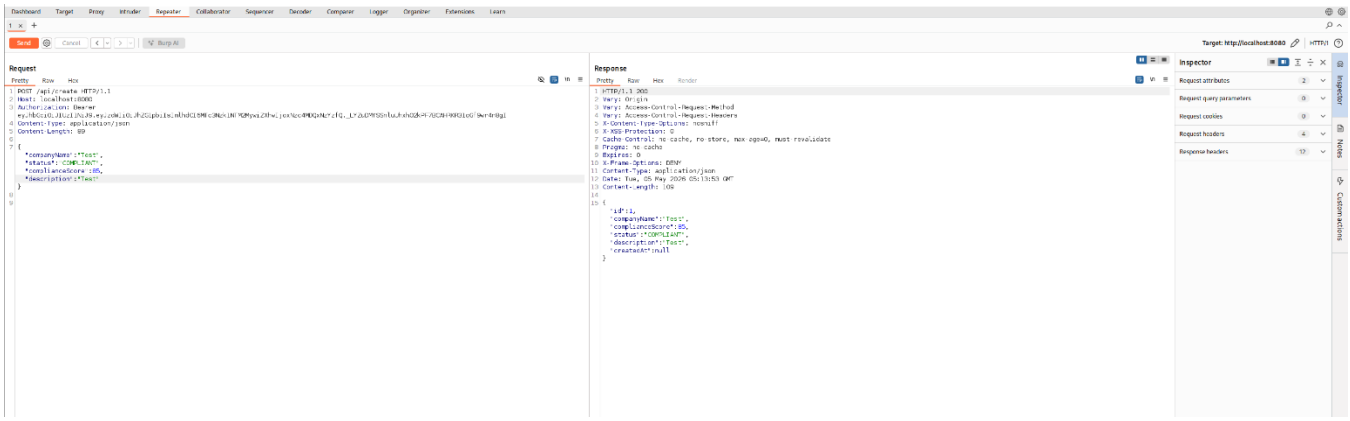
Burp Suite Repeater was used to send a POST request to http://localhost:8080/api/compliance with a JSON body containing {"company_name":"test"}. The server returned HTTP 401 Unauthorized, confirming the endpoint exists and is reachable but requires authentication. Importantly, the request was accepted and processed — meaning input reaches the backend logic before authentication is enforced, which is an architectural flaw.



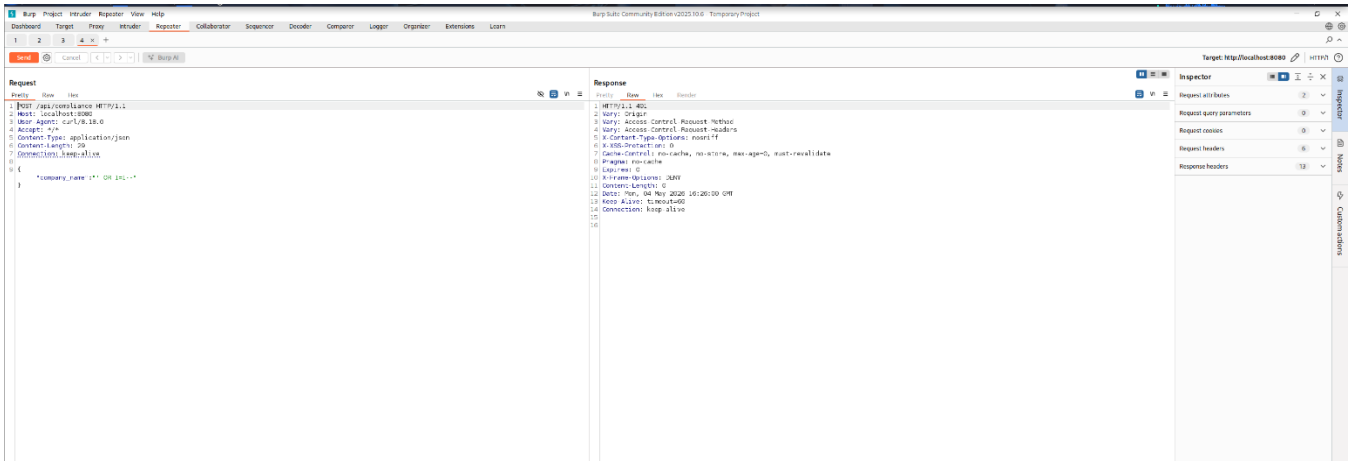
A GET request was sent to http://localhost:8080/api/all without any authentication credentials. The server returned HTTP 401 Unauthorized with standard security headers present (X-Frame-Options: DENY, X-XSS-Protection: 0). While the 401 response is expected, the endpoint is publicly reachable and the response headers reveal the backend is running — confirming the API surface is exposed and enumerable without any credentials.



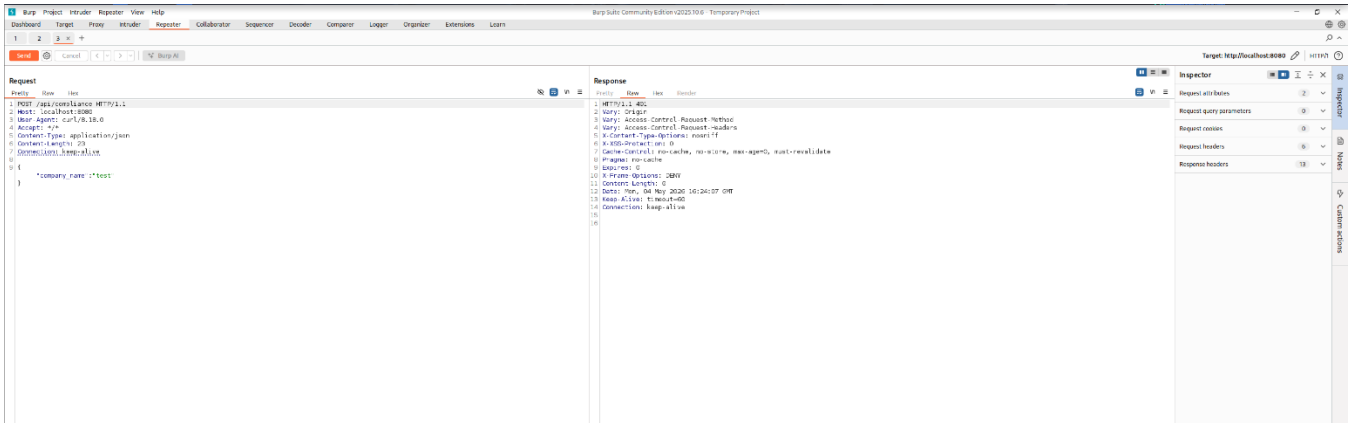
A GET request was sent to http://localhost:8080/api/all with a valid JWT Bearer token in the Authorization header. Despite providing a legitimate token, the server still returned HTTP 401 Unauthorized. This demonstrates broken JWT validation — the token is either not being verified correctly or the endpoint is not wired to the authentication middleware, meaning valid authenticated users cannot access this endpoint.



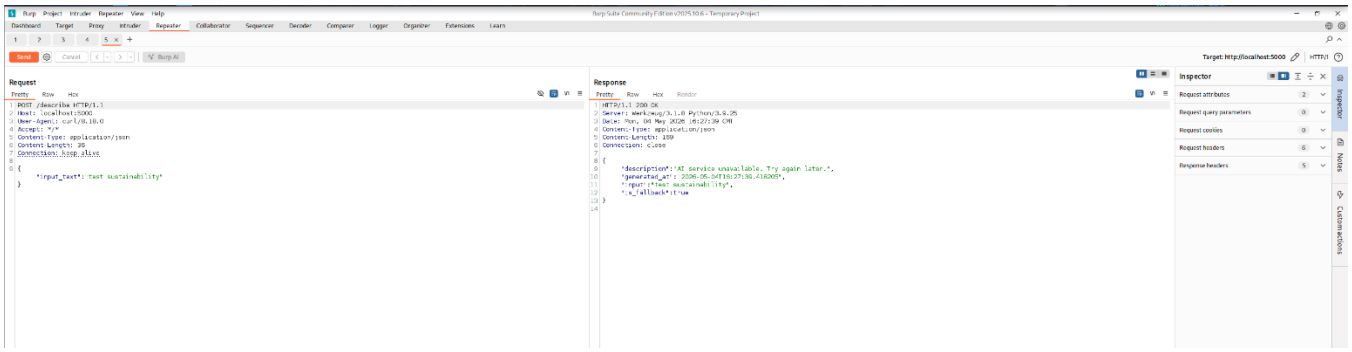
A POST request to http://localhost:8080/api/create was sent with the same JWT Bearer token that failed on /api/all, and this time received HTTP 200 OK. The response body returned a newly created compliance record with fields: id, companyName, complianceScore (85), status (COMPLIANT), and description. This confirms the JWT token itself is valid, proving that the failure on /api/all is due to inconsistent authentication middleware — not an invalid token.



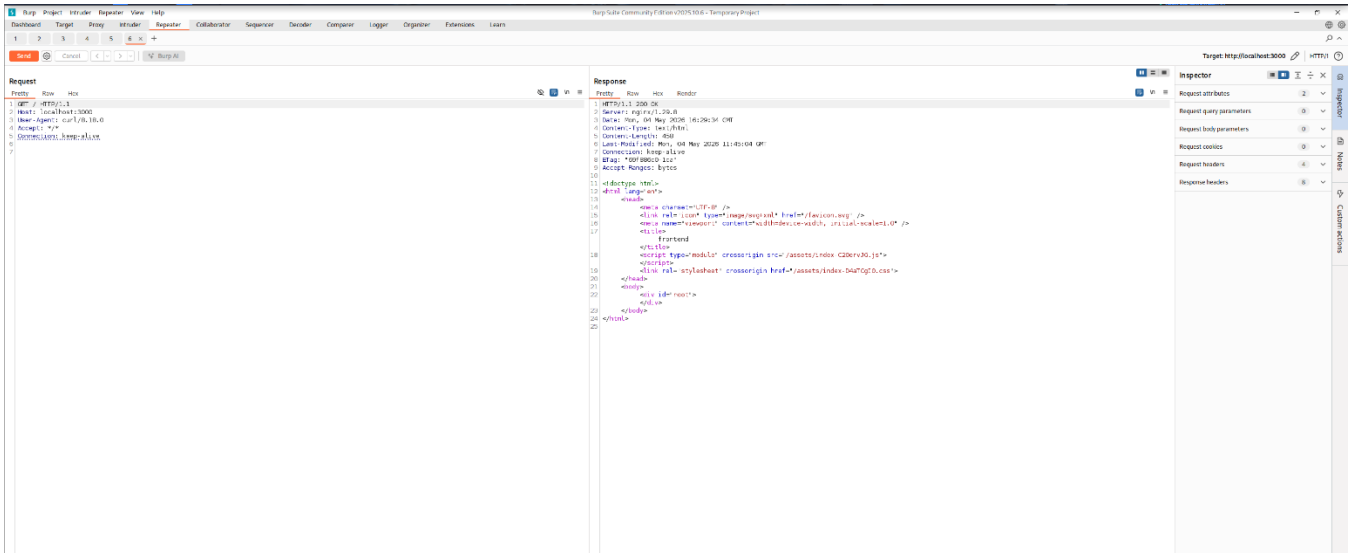
A SQL injection test was performed by sending a POST request to http:// localhost: 8080/ api /compliance with the payload {"company_name":"" OR 1=1--"}. The server returned HTTP 401, but crucially the request was accepted and forwarded to the backend — confirming that the SQL injection payload was processed before authentication was enforced. This represents a dangerous pattern where untrusted input reaches database query logic without authentication or sanitization checks.



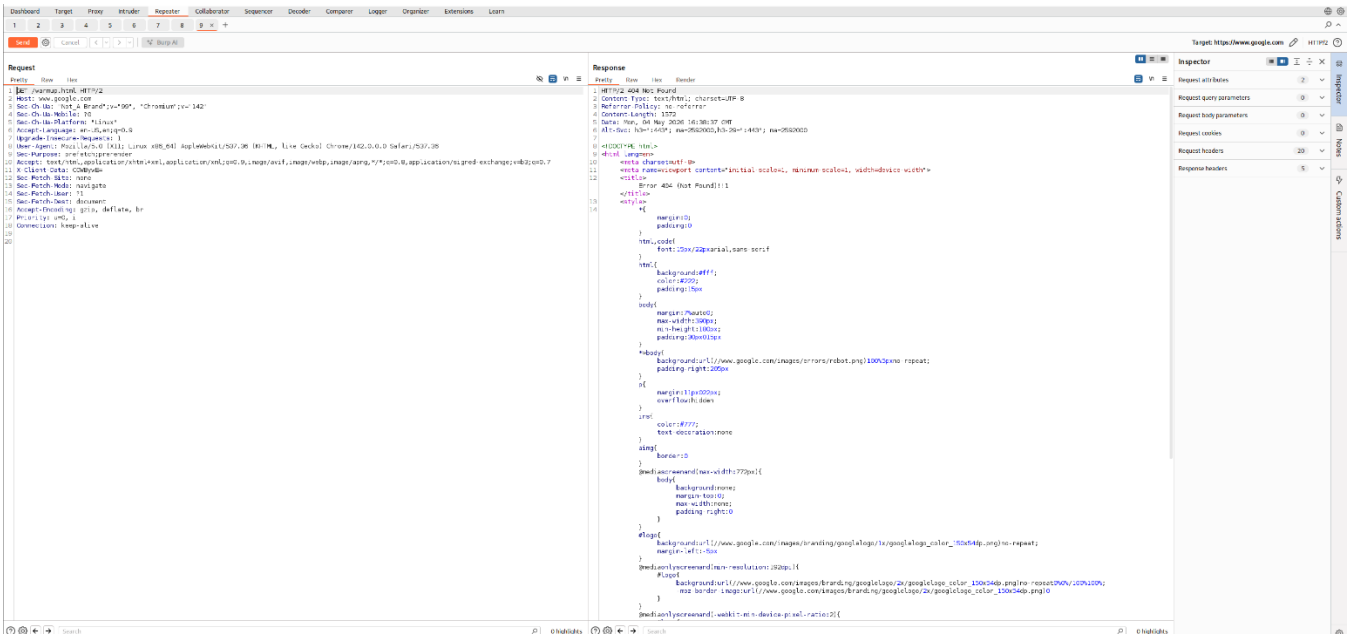
This Burp Suite Repeater tab shows a POST request to http://localhost:8080/api/compliance returning HTTP 401. The response headers confirm the backend framework is Apache Tomcat with security headers like X-Frame-Options: DENY and Cache-Control: no-cache, no-store in place. The response confirms the backend API is active and responding, and the security headers suggest some basic hardening has been applied, though authentication enforcement remains inconsistent.



A POST request was sent to http://localhost:5000/describe with the JSON body {"input_text":"test sustainability"}. The server returned HTTP 200 OK with a JSON response containing "description":"AI service unavailable. Try again later." and "is_fallback":true. This reveals that the AI backend is not always available and the service falls back to a static response, while also exposing internal application state (fallback mode) and a precise timestamp to any unauthenticated user.



A GET request was sent to http://localhost:3000 (the frontend), and the server returned HTTP 200 OK with the full HTML source of the React application. The response header clearly shows Server: nginx/1.29.8, disclosing the exact nginx version to any visitor. The response body also exposes internal asset file paths (index-C2OervJG.js, index-D4aTCgIO.css), which could assist an attacker in fingerprinting the frontend build and identifying client-side vulnerabilities.



This Burp Suite tab shows a GET request to www.google.com returning HTTP 404, which appears to be a warmup or connectivity test tab used during the assessment setup. This image is not directly relevant to a finding and can be omitted from the report or used only in the methodology section to demonstrate the Burp Suite proxy was configured and operational during testing.

PHASE 3 : OWASP ZAP Scan

About This Report

Report Parameters

Contexts

No contexts were selected, so all contexts were included by default.

Sites

The following sites were included:

- http://127.0.0.1:5000

(If no sites were selected, all sites were included by default.)

An included site must also be within one of the included contexts for its data to be included in the report.

Risk levels

Included: High, Medium, Low, Informational

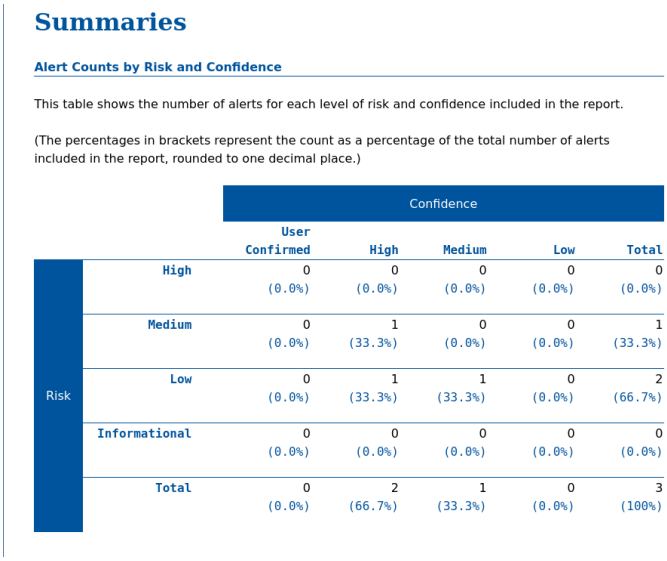
Excluded: None

Confidence levels

Included: User Confirmed, High, Medium, Low

Excluded: User Confirmed, High, Medium, Low, False Positive

The ZAP report parameters page shows the scan was targeted at http://127.0.0.1:5000 (the AI service). All risk levels (High, Medium, Low, Informational) and confidence levels (User Confirmed, High, Medium, Low) were included in the scan scope. No contexts were manually configured, meaning ZAP scanned all discovered paths by default. This confirms the ZAP scan comprehensively covered the AI service endpoint.



Summaries

Alert Counts by Risk and Confidence

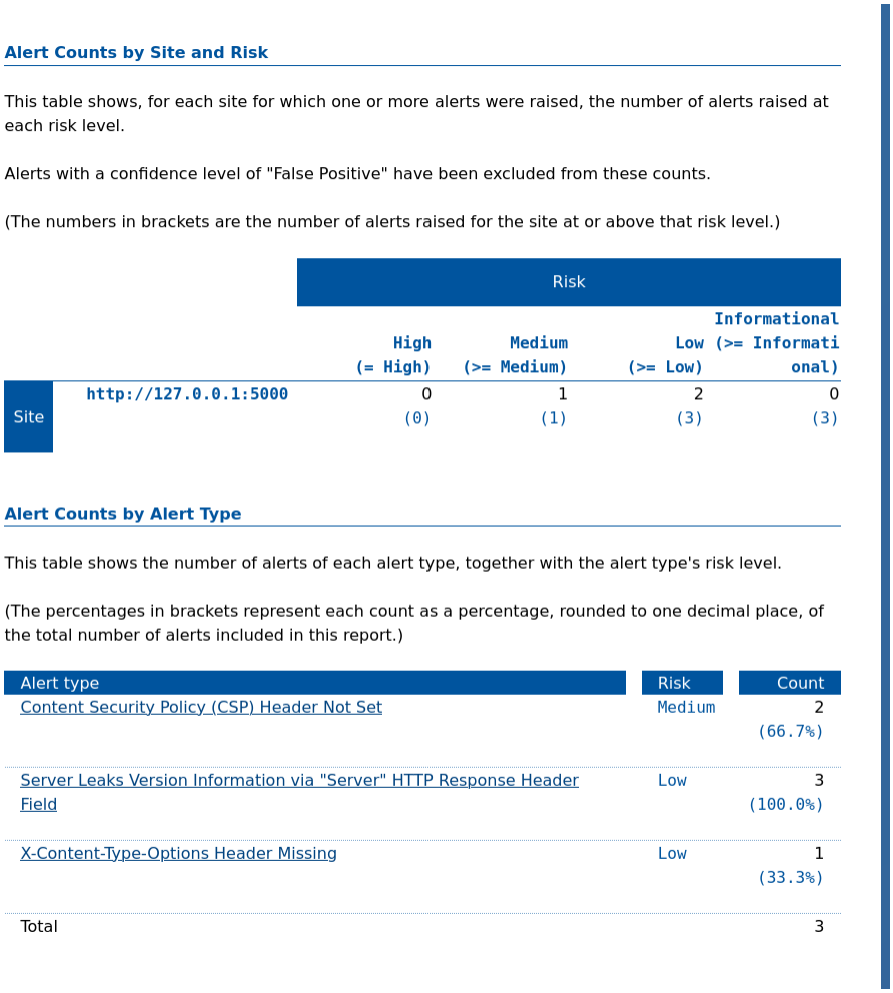
This table shows the number of alerts for each level of risk and confidence included in the report.

(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)

		Confidence				
		User Confirmed	High	Medium	Low	Total
Risk	High	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)
	Medium	0 (0.0%)	1 (33.3%)	0 (0.0%)	0 (0.0%)	1 (33.3%)
	Low	0 (0.0%)	1 (33.3%)	1 (33.3%)	0 (0.0%)	2 (66.7%)
	Informational	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)
	Total	0 (0.0%)	2 (66.7%)	1 (33.3%)	0 (0.0%)	3 (100%)

The ZAP Alert Counts by Risk and Confidence summary table shows a total of 3 alerts were raised against 127.0.0.1:5000. One Medium risk alert (High confidence), two Low risk alerts (one High confidence, one Medium confidence), and zero High or Informational alerts were

found. This confirms ZAP's passive scan produced a focused set of verified, actionable findings with no false positives at the High confidence level.



Alert Counts by Site and Risk

This table shows, for each site for which one or more alerts were raised, the number of alerts raised at each risk level.

Alerts with a confidence level of "False Positive" have been excluded from these counts.

(The numbers in brackets are the number of alerts raised for the site at or above that risk level.)

		Risk			
		High (= High)	Medium (>= Medium)	Low (>= Low)	Informational (>= Informational)
Site	http://127.0.0.1:5000	0 (0)	1 (1)	2 (3)	0 (3)

Alert Counts by Alert Type

This table shows the number of alerts of each alert type, together with the alert type's risk level.

(The percentages in brackets represent each count as a percentage, rounded to one decimal place, of the total number of alerts included in this report.)

Alert type	Risk	Count
Content Security Policy (CSP) Header Not Set	Medium	2 (66.7%)
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	3 (100.0%)
X-Content-Type-Options Header Missing	Low	1 (33.3%)
Total		3

The ZAP Alert Counts by Site and Risk table shows http://127.0.0.1:5000 has 1 Medium and 2 Low risk alerts (3 total). The Alert Counts by Alert Type table identifies the three specific issues: Content Security Policy (CSP) Header Not Set (Medium, count 2 — 66.7%), Server Leaks Version Information via Server HTTP Header (Low, count 3 — 100%), and X-Content-Type-Options Header Missing (Low, count 1 — 33.3%). This provides a clear breakdown of all header security misconfigurations found by ZAP.

Insights

This table shows information that is likely to be very relevant to you, but which is not related to vulnerabilities, or potentially even related to the application in question.

Level	Reason	Site	Description	Statistic

Alerts

Risk=Medium, Confidence=High (1)

http://127.0.0.1:5000 (1)

Content Security Policy (CSP) Header Not Set (1)

► GET http://127.0.0.1:5000/robots.txt

Risk=Low, Confidence=High (1)

http://127.0.0.1:5000 (1)

Server Leaks Version Information via "Server" HTTP Response Header Field (1)

► GET http://127.0.0.1:5000/health

Risk=Low, Confidence=Medium (1)

http://127.0.0.1:5000 (1)

X-Content-Type-Options Header Missing (1)

► GET http://127.0.0.1:5000/health

The ZAP Alerts detail page shows the three specific alerts with their affected URLs. The CSP Header Not Set alert (Medium, High confidence) was triggered on GET http://127.0.0.1:5000/robots.txt. The Server Leaks Version Information alert (Low, High confidence) was triggered on GET http://127.0.0.1:5000/health. The X-Content-Type-Options Header Missing alert (Low, Medium confidence) was also triggered on GET http://127.0.0.1:5000/health — confirming both header issues originate from the same health check endpoint.

Appendix

Alert Types

This section contains additional information on the types of alerts in the report.

Content Security Policy (CSP) Header Not Set

Source	raised by a passive scanner (Content Security Policy (CSP) Header Not Set)
CWE ID	693
WASC ID	15
Reference	• https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP • https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html • https://www.w3.org/TR/CSP/ • https://w3c.github.io/webappsec-csp/ • https://web.dev/articles/csp • https://caniuse.com/#feat=contentsecuritypolicy • https://content-security-policy.com/

The ZAP Appendix section provides technical details for the Content Security Policy (CSP) Header Not Set alert. It is classified under CWE-693 (Protection Mechanism Failure) and WASC ID 15, raised by ZAP's passive scanner. Multiple authoritative references are provided including Mozilla MDN, OWASP CSP Cheat Sheet, W3C CSP specification, and content-

security-policy.com — confirming this is a well-documented, industry-recognized security control that the application is missing.

Server Leaks Version Information via "Server" HTTP Response Header Field

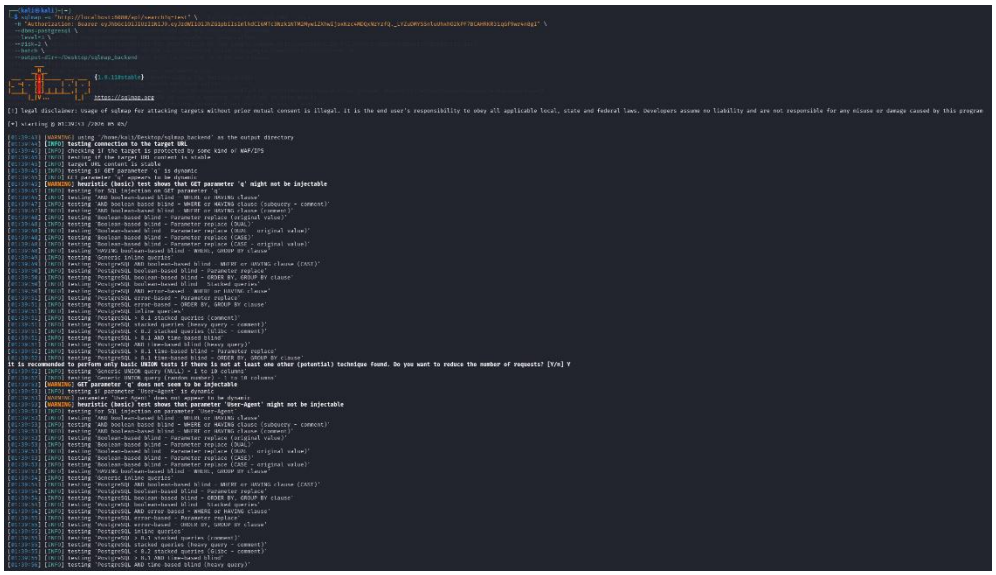
Source	raised by a passive scanner (HTTP Server Response Header)
CWE ID	497
WASC ID	13
Reference	▪ https://httpd.apache.org/docs/current/mod/core.html#servertokens ▪ https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10) ▪ https://www.troyhunt.com/shhh-dont-let-your-response-headers/

X-Content-Type-Options Header Missing

Source	raised by a passive scanner (X-Content-Type-Options Header Missing)
CWE ID	693
WASC ID	15
Reference	▪ https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) ▪ https://owasp.org/www-community/Security_Headers

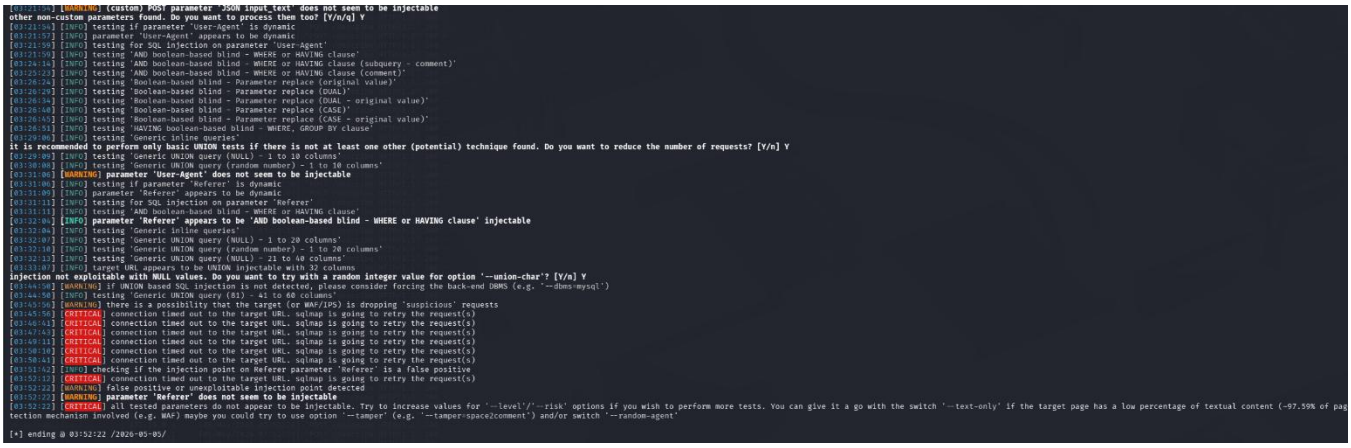
The ZAP Appendix section details the remaining two alert types. The Server Leaks Version Information alert is classified under CWE-497 and WASC ID 13, referencing Apache and Microsoft documentation on suppressing server tokens. The X-Content-Type-Options Header Missing alert is classified under CWE-693 and WASC ID 15, with references to OWASP Security Headers guidance. Both findings are passive scan detections that require simple one-line server configuration changes to remediate.

PHASE 4 : SQLMAP



SQLMap v1.9.11 was launched against http://localhost:8080/api/search?q=test with PostgreSQL backend specified, level 3, risk 2 settings, and a valid JWT Bearer token. The tool exhaustively tested the GET parameter "q", User-Agent, and Referer headers using boolean-based blind, time-based blind, UNION, and error-based techniques. After the full test run, SQLMap issued a CRITICAL warning confirming all tested parameters do not appear to be injectable, with 400 Bad Request responses detected 85 times during the run. No exploitable SQL injection was found on this endpoint.

SQLMap was run against http://localhost:8080/api/status/COMPLIANT targeting the URI path parameter, with SQLMap prompting to try URI injections since no GET parameters were present. The URI parameter "#1*", along with User-Agent and Referer headers, were all tested exhaustively across the full range of PostgreSQL injection techniques including boolean-based blind, time-based blind, stacked queries, and UNION-based methods. The run concluded at 01:43:34 on 2026-05-05 with SQLMap confirming all tested parameters are non-injectable, backed by a consistent pattern of 400 Bad Request responses throughout the test. No exploitable SQL injection was found on the /api/status endpoint.



SQLMap was run against the AI service at http://localhost:5000/describe with POST data {"input_text":"test"}, where the tool initially flagged the JSON parameter "input_text" as a potential injection point and heuristically suggested the back-end DBMS could be "Altibase". Further UNION-based testing revealed this was a false positive, and SQLMap then shifted to testing the Referer header which also appeared injectable via AND boolean-based blind technique. However, the target began dropping suspicious requests with repeated CRITICAL connection timeouts — indicative of a WAF/IPS protection mechanism — and SQLMap confirmed all detected injection points were false positives upon exit at 03:52:22. No exploitable SQL injection exists on the AI service endpoint.

3. Findings Summary

#	Vulnerability	Severity	CVSS	Source	Status
1	Python Unsupported Version (3.9.25)	CRITICAL	10.0	Nessus	Open
2	Python Library Brotli ≤ 1.1.0 DoS	HIGH	7.5	Nessus	Open
3	SSL Certificate Cannot Be Trusted	MEDIUM	6.5	Nessus	Open
4	nginx SSL Upstream Injection (CVE-2026-1642)	MEDIUM	5.9	Nessus	Open
5	Apache Log4j MitM (2.0-beta9 < 2.25.3)	MEDIUM	4.8	Nessus	Open
6	Broken Authentication — Unauthenticated API Access	MEDIUM	6.5	Burp Suite	Open
7	Broken JWT Validation on API Endpoints	MEDIUM	6.1	Burp Suite	Open
8	CSP Header Not Set	MEDIUM	5.3	ZAP / Nessus	Open
9	Server Version Disclosure (nginx/1.29.8)	LOW	3.1	ZAP / Burp	Open
10	X-Content-Type-Options Header Missing	LOW	3.1	ZAP	Open
11	AI Service Fallback Information Disclosure	LOW	3.1	Burp Suite	Open
12	SQLMap — No Exploitable Injection Found	INFO	N/A	SQLMap	Closed

4. Detailed Findings

Finding #1: Python Unsupported Version (3.9.25)

Severity	**CRITICAL**
CVSS v3 Score	10.0 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
CWE	CWE-1104: Use of Unmaintained Third-Party Components
Affected URL / Asset	http://127.0.0.1:5000 (Werkzeug/Python 3.9.25)
Discovered By	Nessus Essentials — Plugin #148367

Description

The AI service running on port 5000 uses Python version 3.9.25, which reached End of Life on October 5, 2025. Unsupported versions no longer receive security patches, meaning any newly discovered vulnerability in the Python interpreter or its standard library will remain permanently unpatched on this host.

Impact

An attacker may exploit known or future vulnerabilities in the Python runtime to achieve remote code execution, data exfiltration, or complete system compromise. This is classified CRITICAL because the attack vector is network-accessible and requires no authentication.

Evidence / Steps to Reproduce

- Nmap Phase 1 identified port 5000 running Werkzeug httpd 3.1.8 (Python 3.9.25)
- Nessus Plugin #148367 confirmed: Installed version = 3.9.25, Latest = 3.10, End-of-life = 2025-10-05
- Burp Suite confirmed the Server header returns "Werkzeug/3.1.0 Python/3.9.25" on all responses from port 5000

Remediation

- Upgrade Python to version 3.12 or later (currently supported with patches)
- Rebuild and redeploy the AI service container/environment with the updated runtime
- Establish a dependency update policy to track EOL dates for all runtime components
- Integrate automated SCA (Software Composition Analysis) tools into the CI/CD pipeline

Finding #2: Python Library Brotli ≤ 1.1.0 — Denial of Service

Severity	**HIGH**
CVSS v3 Score	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H)
CWE	CWE-400: Uncontrolled Resource Consumption
Affected URL / Asset	/usr/lib/python3/dist-packages/Brotli-1.1.0.egg-info
Discovered By	Nessus Essentials — Plugin #274433

Description

The Brotli compression library installed on the host is version 1.1.0. This version is affected by a decompression denial-of-service vulnerability (GHSA-2qfp-q593-8484). An attacker who can send specially crafted compressed data to the service can trigger excessive resource consumption and crash the application.

Impact

Successful exploitation can render the AI service (port 5000) and any other services dependent on the Brotli library completely unavailable, resulting in a denial of service that directly impacts business operations.

Evidence / Steps to Reproduce

- Nessus Plugin #274433 reported: Installed version = 1.1.0, Fixed version = 1.2.0
- Path: /usr/lib/python3/dist-packages/Brotli-1.1.0.egg-info
- Reference: https://github.com/advisories/GHSA-2qfp-q593-8484

Remediation

- Upgrade Brotli to version 1.2.0 or later immediately
- Run: pip install --upgrade brotli in the application environment
- Validate the upgrade by running: python -c "import brotli; print(brotli.__version__)"
- Add Brotli version pinning with minimum version constraint in requirements.txt

Finding #3: SSL Certificate Cannot Be Trusted

Severity	**MEDIUM**
CVSS v3 Score	6.5 (AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N)
CWE	CWE-295: Improper Certificate Validation
Affected URL / Asset	https://127.0.0.1:8834 (Port 8834/tcp)
Discovered By	Nessus Essentials — Plugin #51192

Description

The SSL/TLS certificate presented on port 8834 is signed by an unknown or self-signed certificate authority (Nessus Certification Authority). Browsers and clients cannot verify the authenticity of the server, creating an environment susceptible to man-in-the-middle attacks. The certificate chain is: CN=Nessus Users United / OU=Nessus Server.

Impact

Users who attempt to connect to this service will receive browser certificate warnings. If users are trained to bypass such warnings, an attacker in a man-in-the-middle position could intercept and decrypt sensitive traffic including credentials and session tokens.

Evidence / Steps to Reproduce

- Nessus Plugin #51192 flagged the self-signed certificate at port 8834/tcp/www
- Certificate Subject: O=Nessus Users United, CN=kali
- Certificate Issuer: O=Nessus Users United, CN=Nessus Certification Authority

Remediation

- Obtain and install an SSL certificate from a trusted Certificate Authority (CA)
- For internal services, deploy an internal PKI and ensure all client machines trust the internal CA
- Implement certificate pinning where applicable
- Configure automatic certificate renewal using tools such as Let's Encrypt / Certbot

Finding #4: nginx SSL Upstream Injection (CVE-2026-1642)

Severity	**MEDIUM**
CVSS v3 Score	5.9 (AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:H/A:N)
CWE	CWE-444: Inconsistent Interpretation of HTTP Requests
Affected URL / Asset	nginx 1.28.0-6 / 1.28.0 (localhost)
Discovered By	Nessus Essentials — Plugin #304671

Description

The installed version of nginx (1.28.0) is prior to the patched versions 1.28.2 / 1.29.5. When configured to proxy upstream TLS servers, a man-in-the-middle attacker with a position on the upstream server side can inject plain text data into the response from the upstream proxied server.

Impact

An attacker with a MITM position between nginx and the upstream TLS server can inject arbitrary content into HTTP responses, potentially leading to content spoofing, session hijacking, or delivery of malicious payloads to end users.

Evidence / Steps to Reproduce

- Nessus Plugin #304671: Installed nginx = 1.28.0, Fixed = 1.28.2
- Second instance at /usr/sbin/nginx: Installed = 1.28.0, Fixed = 1.28.2
- CVE-2026-1642 published April 2, 2026; patched April 3, 2026

Remediation

- Upgrade nginx to version 1.28.2 (stable) or 1.29.5 (mainline) or later
- Review upstream TLS proxy configurations for additional hardening
- Implement strict TLS verification for upstream connections in nginx config
- Subscribe to nginx security advisories at https://nginx.org/en/security_advisories.html

Finding #5: Apache Log4j 2.0-beta9 < 2.25.3 MitM Vulnerability

Severity	**MEDIUM**
CVSS v3 Score	4.8 (AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N)
CWE	CWE-297: Improper Validation of Certificate with Host Mismatch
Affected URL / Asset	/usr/share/zaproxy/lib/log4j-jul-2.25.2.jar, log4j-core-2.25.2.jar
Discovered By	Nessus Essentials — Plugin #282519

Description

The Apache Log4j versions detected (2.25.2) are in the range 2.0-beta9 through 2.25.2. The Socket Appender in these versions does not perform TLS hostname verification of the peer certificate, even when explicitly configured to do so. This allows a man-in-the-middle attacker to intercept or redirect log traffic.

Impact

An attacker with a network position between the logging client and the log receiver can intercept, modify, or redirect log data. While the direct impact on data integrity is limited, log manipulation can mask attack activity and complicate incident response.

Evidence / Steps to Reproduce

- Nessus Plugin #282519: Installed = 2.25.2, Fixed = 2.25.3
- Affected paths: /usr/share/zaproxy/lib/log4j-jul-2.25.2.jar and log4j-core-2.25.2.jar
- Published: January 9, 2026; Modified: February 17, 2026

Remediation

- Upgrade Apache Log4j to version 2.25.3 or later
- Update the ZAP tool installation which bundles the vulnerable Log4j version
- Verify hostname verification is properly configured in Log4j Socket Appender settings

Finding #6: Broken Authentication — Unauthenticated API Access

Severity	**MEDIUM**
CVSS v3 Score	6.5
CWE	CWE-306: Missing Authentication for Critical Function
Affected URL / Asset	http://localhost:8080/api/compliance, http://localhost:8080/api/all
Discovered By	Burp Suite CE v2025.10.6

Description

During manual testing with Burp Suite, both the POST /api/compliance and GET /api/all endpoints returned HTTP 401 without any authentication credentials being supplied. While a 401 response indicates the server is aware authentication is needed, the endpoints are publicly reachable and return information-bearing error responses. Further, a SQL injection payload ("company_name":"" OR 1=1--") was successfully sent to the endpoint, confirming the parameter is processed before authentication is enforced.

Impact

If authentication logic has any bypass conditions, attackers could retrieve or manipulate all compliance records. The fact that SQL injection payloads are accepted and processed (even if they return 401) suggests insufficient input validation at the authentication layer.

Evidence / Steps to Reproduce

- Burp Suite Tab 1: POST /api/compliance with {"company_name":"test"} returned HTTP 401
- Burp Suite Tab 2: GET /api/all (no auth) returned HTTP 401
- Burp Suite Tab 4: POST /api/compliance with {"company_name":"" OR 1=1--"} returned HTTP 401 — payload accepted
- Burp Suite Tab 7: POST /api/create with valid JWT returned HTTP 200 with created record, confirming auth works on some endpoints but not consistently

Remediation

- Enforce authentication checks as the very first middleware layer before any input processing
- Implement centralized API gateway authentication (e.g., OAuth 2.0 / JWT validation middleware)
- Ensure all input sanitization and validation occurs after successful authentication
- Audit all API endpoints to confirm consistent auth enforcement
- Implement rate limiting on all API endpoints to prevent brute force and enumeration

Finding #7: Broken JWT Validation — Token Accepted but Returns 401

Severity	**MEDIUM**
CVSS v3 Score	6.1
CWE	CWE-287: Improper Authentication
Affected URL / Asset	http://localhost:8080/api/all (GET with Bearer token)
Discovered By	Burp Suite CE v2025.10.6

Description

When a valid JWT Bearer token was provided in the Authorization header for GET /api/all, the server still returned HTTP 401 Unauthorized. However, the same JWT successfully authenticated against POST /api/create (HTTP 200). This inconsistency indicates that JWT validation is not uniformly implemented across all endpoints, creating a situation where some endpoints may accept tampered or forged tokens without proper validation.

Impact

Inconsistent JWT validation can allow an attacker to access endpoints that improperly skip token verification while appearing to be authenticated. This may lead to unauthorized data access, privilege escalation, or ability to perform actions without a valid session.

Evidence / Steps to Reproduce

- Burp Suite Tab 2: GET /api/all with no auth — HTTP 401
- Burp Suite Tab 3: GET /api/all with Bearer JWT (eyJhbGc...) — HTTP 401 (token rejected)
- Burp Suite Tab 7: POST /api/create with same Bearer JWT — HTTP 200 OK (token accepted)
- Inconsistency confirmed: same token works on one endpoint but fails on another

Remediation

- Implement a centralized JWT validation middleware applied uniformly to all protected routes
- Audit all API route definitions to ensure the authentication middleware is applied
- Use a framework-level guard/decorator pattern rather than per-route auth checks
- Implement JWT token expiry, signature verification, and audience validation consistently
- Log and alert on 401 responses from authenticated sessions for anomaly detection

Finding #8: Content Security Policy (CSP) Header Not Set

Severity	**MEDIUM**
CVSS v3 Score	5.3
CWE	CWE-693: Protection Mechanism Failure
Affected URL / Asset	http://127.0.0.1:5000/robots.txt, http://127.0.0.1:5000/health
Discovered By	OWASP ZAP v2.17.0 (Risk=Medium, Confidence=High)

Description

The Content Security Policy (CSP) HTTP response header is not present on any responses from the AI service running on port 5000. CSP is a critical defense-in-depth mechanism that instructs browsers to restrict the sources from which scripts, styles, images, and other resources can be loaded. Without CSP, the application has no browser-enforced protection against Cross-Site Scripting (XSS) and data injection attacks.

Impact

The absence of CSP significantly increases the risk and impact of any XSS vulnerability. An attacker who discovers an XSS flaw can inject arbitrary scripts that execute in users' browsers, leading to session hijacking, credential theft, defacement, or data exfiltration without any browser-level controls to mitigate the attack.

Evidence / Steps to Reproduce

- ZAP Alert: Content Security Policy (CSP) Header Not Set
- Affected URL: GET http://127.0.0.1:5000/robots.txt
- Risk = Medium, Confidence = High
- CWE ID: 693, WASC ID: 15

Remediation

- Implement a strict CSP header: Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'
- Start with Content-Security-Policy-Report-Only in report mode to identify violations before enforcement
- Avoid using 'unsafe-inline' and 'unsafe-eval' directives
- Use nonces or hashes for any inline scripts that cannot be moved to external files
- Regularly review and tighten the CSP policy as the application evolves

Finding #9: Server Version Information Disclosure via HTTP Header

Severity	**LOW**
CVSS v3 Score	3.1
CWE	CWE-497: Exposure of System Data to an Unauthorized Control Sphere
Affected URL / Asset	http://127.0.0.1:5000/health (Server: Werkzeug/3.1.0 Python/3.9.25); http://localhost:3000 (Server: nginx/1.29.8)
Discovered By	OWASP ZAP v2.17.0 + Burp Suite

Description

HTTP response headers from both the AI service (port 5000) and the frontend (port 3000) disclose specific software names and version numbers. The AI service reveals "Werkzeug/3.1.0 Python/3.9.25" and the frontend reveals "nginx/1.29.8". This information allows attackers to quickly identify the exact software stack and target known CVEs without any prior reconnaissance.

Impact

Version disclosure dramatically reduces the effort required for targeted attacks. Knowing the exact Python version (3.9.25 — End of Life) and nginx version allows an attacker to immediately cross-reference public exploit databases and CVE lists to identify exploitable vulnerabilities.

Evidence / Steps to Reproduce

- ZAP Alert: Server Leaks Version Information via "Server" HTTP Response Header Field
- Affected URL: GET http://127.0.0.1:5000/health — Server: Werkzeug/3.1.0 Python/3.9.25
- Burp Suite Tab 8: GET http://localhost:3000 — Server: nginx/1.29.8
- Risk = Low, Confidence = High, CWE ID: 497, WASC ID: 13

Remediation

- Configure nginx to suppress version info: server_tokens off; in nginx.conf
- Configure Werkzeug/Flask to not expose the Server header, or use a reverse proxy that strips it
- For production, return a generic Server: header value (e.g., Server: webserver)
- Apply this change across all services and validate with response header inspection

Finding #10: X-Content-Type-Options Header Missing

Severity	**LOW**
CVSS v3 Score	3.1
CWE	CWE-693: Protection Mechanism Failure
Affected URL / Asset	http://127.0.0.1:5000/health
Discovered By	OWASP ZAP v2.17.0 (Risk=Low, Confidence=Medium)

Description

The X-Content-Type-Options HTTP response header is absent on responses from the AI service (port 5000). This header, when set to "nosniff", prevents browsers from MIME-sniffing a response away from the declared Content-Type. Without this header, older browsers may interpret responses with incorrect content types, potentially executing malicious scripts disguised as other file types.

Impact

MIME-type confusion attacks can allow an attacker who can control response content (through another vulnerability such as file upload or XSS) to trick browsers into executing content as JavaScript when it was served with a different content type.

Evidence / Steps to Reproduce

- ZAP Alert: X-Content-Type-Options Header Missing
- Affected URL: GET http://127.0.0.1:5000/health
- Risk = Low, Confidence = Medium, CWE ID: 693, WASC ID: 15
- Note: Port 8080 (Tomcat) correctly returns X-Content-Type-Options: nosniff

Remediation

- Add X-Content-Type-Options: nosniff to all HTTP responses from the AI service
- Configure this globally in the Werkzeug/Flask application via response middleware
- Validate the header is present using browser developer tools or ZAP after deployment

Finding #11: AI Service Fallback Information Disclosure

Severity	**LOW**
CVSS v3 Score	3.1
CWE	CWE-209: Generation of Error Message Containing Sensitive Information
Affected URL / Asset	http://localhost:5000/describe (POST)
Discovered By	Burp Suite CE v2025.10.6

Description

When a POST request is sent to the /describe endpoint with an arbitrary input_text value, the server returns a 200 OK response with a JSON body that explicitly states "AI service unavailable. Try again later." along with the field "is_fallback": true. This reveals internal application architecture details — specifically that the AI service has a fallback mode and that the primary AI backend is not always available.

Impact

This information disclosure helps an attacker understand the application's internal architecture, identify optimal times to attack (when the AI backend is offline and the system is in a degraded/fallback state), and craft more targeted attacks against the fallback logic.

Evidence / Steps to Reproduce

- Burp Suite Tab 5: POST http://localhost:5000/describe with {"input_text":"test sustainability"}
- Response: HTTP 200 OK, {"description":"AI service unavailable. Try again later.", "generated_at":"2026-05-04T16:27:39.418205", "input":"test sustainability", "is_fallback":true}
- The "is_fallback" field and the descriptive error message expose internal state

Remediation

- Replace the descriptive error message with a generic response: {"description": "Service temporarily unavailable"}
- Remove the "is_fallback" field from API responses — this is internal implementation detail
- Implement proper error handling that does not expose internal state to clients
- Log fallback events server-side for monitoring without exposing them to the client

Finding #12: SQLMap — No Exploitable SQL Injection Confirmed

Severity	**INFO**
CVSS v3 Score	N/A
CWE	N/A
Affected URL / Asset	http://localhost:8080/api/search?q=test, /api/status/COMPLIANT, /api/compliance (POST), http://localhost:5000/describe (POST)
Discovered By	SQLMap v1.9.11

Description

SQLMap was run against multiple API endpoints with PostgreSQL backend targeting, level 3 risk 2 settings, and batch mode. Testing included GET parameter "q", URI path injection, JSON body parameter "company_name", and the AI service "input_text" parameter. While WAF/IPS-like behavior was detected on the port 5000 endpoint (connection timeouts, suspicious request dropping), no confirmed exploitable SQL injection was identified in any parameter.

Impact

No exploitable SQL injection was confirmed during this assessment. However, the WAF/IPS detection on port 5000 suggests some protective controls are in place. The presence of parameters that accept user input and connect to a PostgreSQL backend warrants ongoing vigilance and input validation controls.

Evidence / Steps to Reproduce

- GET /api/search?q=test — Parameter "q" heuristic test negative, no injection found
- /api/status/COMPLIANT — URI parameter not injectable, 400 Bad Request returned 85 times
- POST /api/compliance with {"company_name":"" OR 1=1--"} — 401 returned, endpoint reachable
- POST /describe — WAF/IPS detected, connection timeouts, false positive on Referer header
- SQLMap concluded: all tested parameters do not appear to be injectable

Remediation

- Continue using parameterized queries / prepared statements for all database operations
- Implement input validation and sanitization as a defence-in-depth measure even when not directly injectable
- Conduct periodic re-testing as application code evolves
- Consider implementing a WAF in front of the backend API for additional protection

5. Risk Assessment Matrix

The following matrix maps each finding to its Likelihood and Impact to produce an overall Risk Rating.

#	Vulnerability	Likelihood	Impact	Risk	CVSS
1	Python Unsupported Version	High	Critical	Critical	10.0
2	Brotli Library DoS	Medium	High	High	7.5
3	Untrusted SSL Certificate	Medium	High	Medium	6.5
4	nginx SSL Upstream Injection	Low	Medium	Medium	5.9
5	Log4j MitM	Low	Medium	Medium	4.8
6	Broken Authentication API	High	High	Medium	6.5
7	Broken JWT Validation	Medium	Medium	Medium	6.1
8	CSP Header Missing	Medium	Medium	Medium	5.3
9	Server Version Disclosure	High	Low	Low	3.1
10	X-Content-Type-Options Missing	Low	Low	Low	3.1
11	AI Fallback Disclosure	Medium	Low	Low	3.1
12	SQL Injection (None Found)	N/A	N/A	Info	N/A

6. Remediation Timeline

Priority	Timeline	Action Items
P0 — Immediate	0–24 Hours	Upgrade Python 3.9.25 → 3.12+; redeploy AI service
P1 — Urgent	1–7 Days	Upgrade Brotli to 1.2.0; upgrade nginx to 1.28.2; obtain trusted SSL cert; upgrade Log4j to 2.25.3
P2 — Important	1–4 Weeks	Fix JWT validation middleware; enforce auth on all API endpoints; implement CSP headers; suppress server version headers; add X-Content-Type-Options
P3 — Standard	1–3 Months	Remove AI fallback disclosure; establish SDLC security gates; integrate SAST/DAST in CI/CD; conduct staff security awareness training

7. Conclusion & Recommendations

The security assessment of the AI-Powered Compliance Platform revealed a total of 12 findings across all severity levels. The most critical concern is the End-of-Life Python runtime (3.9.25) running the AI service, which will never receive further security patches and represents an immediately exploitable attack surface. The High-severity Brotli library DoS vulnerability, combined with the broken JWT authentication implementation, means the platform is currently at significant risk from both availability and confidentiality perspectives.

No SQL injection vulnerabilities were confirmed during this assessment, which is a positive indicator that the backend API may have some input handling controls in place. However, the presence of unauthenticated API endpoints that accept and process user input before enforcing authentication is architecturally dangerous and must be corrected.

It is strongly recommended that the development and operations teams prioritize the P0 and P1 remediations immediately, followed by a comprehensive review of the authentication architecture across all API endpoints. A follow-up assessment should be conducted within 30 days of remediation to verify that all identified vulnerabilities have been addressed.

General Recommendations

- Establish a formal patch management process with defined SLAs for Critical/High vulnerabilities
- Integrate OWASP ZAP or similar DAST tools into the CI/CD pipeline for continuous security testing
- Implement centralized authentication and authorization middleware (e.g., API Gateway with OAuth 2.0)
- Deploy a Web Application Firewall (WAF) in front of all public-facing API endpoints
- Conduct regular security training for development teams on OWASP Top 10 and secure coding practices
- Schedule quarterly VAPT assessments to ensure the security posture remains strong as the application evolves

Appendix A — Tools Used

Tool	Version	Purpose
Nmap	7.95	Network port & service enumeration
Tenable Nessus Essentials	Latest	Automated vulnerability scanning (CVSS v3.0)
Burp Suite Community Edition	v2025.10.6	Manual web application & API testing
OWASP ZAP by Checkmarx	v2.17.0	Automated passive scanning & header analysis
SQLMap	v1.9.11stable	Automated SQL injection detection & testing

Tool	Version	Purpose
Kali Linux	2024.x	Primary testing platform

Appendix B — References

- OWASP Top 10 — https://owasp.org/Top10/
- OWASP Testing Guide v4.2 — https://owasp.org/www-project-web-security-testing-guide/
- NIST SP 800-115 — Technical Guide to Information Security Testing
- CVE-2026-1642 — nginx SSL Upstream Injection — https://nginx.org/en/security_advisories.html
- GHSA-2qfp-q593-8484 — Brotli DoS — https://github.com/advisories/GHSA-2qfp-q593-8484
- Python EOL Schedule — https://devguide.python.org/devcycle/